

How to Run the System

A first-to-last guide: prerequisites, database setup, starting both servers, logging in, and shutting everything down.

REAK is two separate programs that run at the same time: a backend API (**REAK.Api**, ASP.NET Core) and a frontend web app (**web**, Next.js). Both need to be running for the site to work. This guide starts them in the right order, from a completely stopped state, all the way to logging in.

Before you start — what needs to be installed

- **.NET 10 SDK** — runs the backend API.
- **Node.js 20 or newer** — runs the frontend.
- **SQL Server** — LocalDB is enough for local use; a full SQL Server instance for anything else.
- **A terminal** — PowerShell or Git Bash both work; this guide shows PowerShell commands.

Check each is installed by running:

```
dotnet --version
node --version
npm --version
```

Step 1 — Open the project

Open a terminal and go to the project's root folder (the one containing `REAK.Api`, `web`, and `REAK.slnx`).

```
cd D:\Real-state
```

Step 2 — Set the backend's configuration

The API needs one environment variable to start at all: a signing key for login sessions. It refuses to start without it — this is intentional, not a bug.

In PowerShell, for the current terminal session only:

```
$env:REAK_JWT_KEY = "<a long random string>"
```

A quick way to generate one: `openssl rand -base64 48` (or any long random text — it just needs to be unpredictable and kept private). The full list of optional settings is in `REAK.Api\.env.example` — copy it to your own local settings if you want to change ports, token lifetimes, or storage location.

Step 3 — Set up the database (first time only)

Skip this step if the database already exists and has data in it — it only needs to be done once, the first time the project is set up on a machine.

3a. Apply the database schema:

```
cd REAK.Api
dotnet ef database update
```

3b. Apply the row-level security rules:

```
sqlcmd -S "(localdb)\MSSQLLocalDB" -d real-estate -i
Data\Security\RowLevelSecurity.sql
```

(Adjust -S if not using LocalDB, and -d if the database has a different name — check ConnectionStrings__DefaultConnection in appsettings.json.)

3c. Verify it worked:

```
sqlcmd -S "(localdb)\MSSQLLocalDB" -d real-estate -i Data\Security\rls_test.sql
```

This runs a self-contained, self-cleaning test — it creates its own throwaway test data, checks 11 security rules, and deletes everything it created. It should print a series of rows ending in 1 or 0 as expected values (documented in docs/TESTING.md).

Step 4 — Start the backend API

From the REAK.Api folder, in the same terminal where you set REAK_JWT_KEY:

```
dotnet run
```

Leave this terminal open and running — closing it stops the API. Wait for a line mentioning Now listening on: http://localhost:5080 before moving on.

Step 5 — Start the frontend

Open a **second** terminal (leave the API running in the first one). Go to the web folder:

```
cd D:\Real-state\web
npm install
```

(The npm install step only needs to be done once, or after dependencies change.)

First time only — copy the example environment file:

```
copy .env.example .env.local
```

The defaults in it already point at http://localhost:5080 (the API) and http://localhost:3000 (itself) — no editing needed for local use.

Then start it:

```
npm run dev
```

Leave this terminal open too. Wait for it to say it's ready, then it's serving at http://localhost:3000.

Step 6 — Open the site and log in

With both terminals still running, open a browser to:

```
http://localhost:3000
```

The public site works immediately with no login. To try the member portal or admin portal, log in at `/login` with one of the test accounts below (development database only — never real credentials):

Account	Password	What it gets you
assoc-admin@example.test	TestPass123!	System admin — full Admin Portal access
kathmandu-prime@example.test	TestPass123!	Member portal — Kathmandu Prime Properties
pokhara-lakeside@example.test	TestPass123!	Member portal — Pokhara Lakeside Realty

Admin Portal is reachable at `/admin` once logged in as the system-admin account above.

Step 7 — Confirm it's actually working

Optional, but a good habit: check both servers are responding before relying on them. In any terminal:

```
curl http://localhost:3000
curl http://localhost:5080/api/reference/property-types
```

Both should return content (not a connection error). If either fails, check that terminal's window for the actual error message — the most common cause is one of the two servers not actually finishing startup yet.

Step 8 — Shutting everything down

Go to each of the two terminals (API and frontend) and press `Ctrl+C`. There's no separate "stop" command — closing or interrupting the terminal stops that process. The database itself keeps its data; nothing needs to be "shut down" on that side.

Troubleshooting

“REAK_JWT_KEY is required” on startup

The environment variable wasn't set in that terminal, or you opened a new terminal without setting it again. Re-run the `$env:REAK_JWT_KEY = ...` command in the same window you run `dotnet run` from.

Frontend loads but every page errors

The API isn't running, or `REAK_API_URL` in `web/.env.local` doesn't match where it's actually listening. Confirm Step 4 finished and shows “Now listening on...”.

Login says invalid credentials for a test account

The database hasn't been seeded, or you're pointed at a different database than expected. Check `ConnectionStrings__DefaultConnection`.

Photos don't show up on a listing

Only relevant to synthetic/test uploads: this build's own QA process once found that a fake, non-image test file was uploaded as a photo in the dev database, which real browsers correctly refuse to render (see `DEVELOPMENT_PLAN.md`, Stage 14). Real uploaded photos render normally.

Where to find more

- `README.md` — project overview and quick links.
- `docs/ARCHITECTURE.md` — how the two apps fit together.
- `docs/DEPLOYMENT.md` — running this in production, not just locally.
- `docs/TESTING.md` — how to reproduce this project's test passes.
- `docs/PRODUCTION-BUILD-REPORT.md` — full feature list, test results, and current production-readiness verdict.